Supplementary Material

RIsearch and siOFF: An integrated, high-performance framework for RNA–RNA interaction and siRNA off-target prediction

Stefano Roncelli[1], Lorenzo Favaro[1], Christian Anthon[1], Jan Gorodkin[1,*]

[1]Center for non-coding RNA in Technology and Health, Section for Health Data Science and AI, Department of Public Health, University of Copenhagen, Thorvaldsensvej 57, 1871 Frederiksberg, Denmark

*Corresponding author. gorodkin@rth.dk

S1 Results

All benchmarks reported here were run on the latest releases of both tools at the time of writing, RIsearch v3.0.0-alpha.4 and siOFF v0.2.0.

S1.1 RIsearch core search benchmark

With RIsearch, we reimplement the seed-and-extend search of RIsearch2 (Alkan et al., 2017) in Rust. To quantify the resulting speed-up, we benchmarked both tools on the original data from the RIsearch2 publication: 63 siRNA guide sequences searched against a chromosome-X target set of Ensembl human cDNA transcripts filtered to ≥ 80 nt, on an AMD EPYC 9554 server. Both tools used pre-built indexes of the same target transcripts and ran with identical search parameters; we compare only search times and exclude index-construction time. Input files were staged on local disk to remove NFS latency, and wall-clock time was recorded with Hyperfine (one warm-up run followed by three timed runs, with the output directory cleared before each run). In all figures, we label the Rust implementation `RIsearch3`.

The parameter grid comprised seed schemes 2:7, 2:7/5, 1:12/6, 1:12/7, 1:12/8 and the singletons 4, 5, 6, 12 and 16; extension lengths 0, 5, 10, 15 and 20 nt; mismatch counts 0–3; and output formats `p1`–`p4`. These seed geometries were chosen because they are the settings commonly used for siRNA and miRNA target searches (Alkan et al., 2017), and the extension lengths, mismatch counts and output formats span the range of options exposed by both tools, so that the comparison reflects typical use rather than a single favourable setting. Thread scaling was measured for seed scheme 6 at 1, 2, 4, 8, 16 and 32 threads. The energy threshold was $-10\,\mathrm{kcal\,mol^{-1}}$ for extension ≥ 5 nt and 0 for seed-only runs (extension < 5 nt, for which no dynamic-programming extension is performed). The two search commands, shown for seed scheme 2:7/5 with extension 20 nt on 32 threads, were:

```
# RIsearch2 (C)
risearch2.x -q sirnas.fa -i chrX.pksuf -s 2:7/5 \
    -e -10 -l 20 --threads 32 -z t99 -p

# RIsearch (Rust; RIsearch3)
risearch search --query sirnas.fa --target chrX.idx --seed 2:7/5 \
    --energy -10 --extension 20 --threads 32 --matrix t99 \
    --no-dedup -p 1 --multifile --compress gzip --output results_dir
```

Option meanings, with the RIsearch2 flag given first and the RIsearch equivalent second:

- `-q sirnas.fa` / `--query sirnas.fa` — FASTA file of the 63 siRNA guide sequences.
- `-i chrX.pksuf` / `--target chrX.idx` — pre-built index of the chromosome-X target set (RIsearch2 suffix-array package file and RIsearch native index, respectively).
- `-s 2:7/5` / `--seed 2:7/5` — seed scheme, specified with identical syntax in both tools: a perfect match within query positions 2–7 with a minimum seed length of 5 nt.
- `-e -10` / `--energy -10` — energy threshold of $-10\,\mathrm{kcal\,mol^{-1}}$; interactions with a hybridisation energy above this value are discarded.
- `-l 20` / `--extension 20` — extension length: the seed alignment is extended by up to 20 nt on each side with the Gotoh dynamic-programming algorithm.
- `--threads 32` / `--threads 32` — number of parallel search threads.
- `-z t99` / `--matrix t99` — energy matrix used for scoring; `t99` selects the Turner 1999 RNA–RNA nearest-neighbour parameters.
- `-p` / `-p 1` — output format; format 1 in RIsearch reproduces the per-interaction output produced by RIsearch2's `-p`.

The remaining RIsearch flags have no RIsearch2 counterpart and were set to reproduce RIsearch2's fixed behaviour, so that both tools perform equivalent work:

- `--no-dedup` — disable deduplication of identical hits, which RIsearch2 does not perform.
- `--multifile` — write one output file per query, matching RIsearch2's per-query output layout.
- `--compress gzip` — gzip-compress the output, matching RIsearch2's gzipped output (RIsearch defaults to zstd, which RIsearch2 lacks; see Fig. S4).
- `--output results_dir` — directory for the output files (RIsearch2 writes to the working directory).

With matched gzip-compressed output, the new RIsearch version runs about twice as fast as RIsearch2 on the canonical seed-and-extend schemes (Fig. S1; median 2.1×, up to 2.4×), the advantage growing with extension length as the Gotoh dynamic-programming stage comes to dominate runtime (Fig. S3) and holding across thread counts (Fig. S2). RIsearch additionally offers zstd-compressed output, which RIsearch2 lacks; substituting zstd (Figs. S2 and S4) widens the margin over RIsearch2 further, most visibly on seed-only searches.

S1.2 Index construction

RIsearch3 builds its suffix-array index in parallel across cores with libsais (Grebnov, 2021), whereas RIsearch2's libdivsufsort construction is single-threaded. Building the GRCh38/hg38 index was benchmarked against RIsearch2 with Hyperfine (Fig. S5). When the index is written to local disk, RIsearch3 is already ∼1.4× faster single-threaded (0.74× the RIsearch2 time) and ∼1.9× faster at 16 threads (0.54×); when the index is instead written to `/dev/null`, the parallel suffix-array construction is up to ∼8.0× faster (0.13× at 16 threads), showing that disk I/O, not computation, dominates the write-to-disk case.

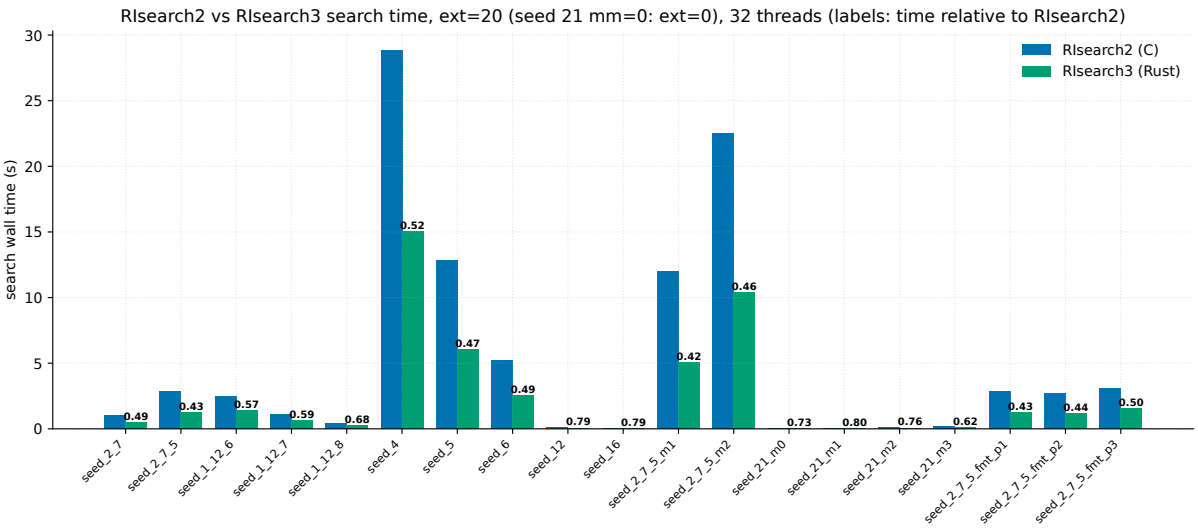


Figure S1: **Per-configuration search time of RIsearch versus RIsearch2.** Wall-clock time at 32 threads with extension length 20 nt (seed 21 mismatch runs: extension 0) and matched gzip-compressed output, across seed schemes, output formats (`fmt_p1`–`p3`) and mismatch counts (`m1`–`m3`). Numbers above each bar pair give RIsearch's time relative to RIsearch2 (lower is faster). On the canonical seed-and-extend schemes RIsearch is roughly twice as fast (relative time 0.41–0.67); the margin narrows for long exact-match singleton seeds (12, 16, 21: 0.58–0.76), whose sub-second runtimes are dominated by fixed start-up rather than by search.

S1.3 siOFF pipeline benchmark

Across batch sizes of 500, 1,000, and 2,000 siRNAs from the genome-wide efficacy library of Huesken et al. (2005), siOFF completes the intersection, accessibility-annotation, partition-function, and output stages 18.1−21.8× faster than the legacy Python 2.7 pipeline (mean wall-clock time 43 s vs. 782 s at 500 siRNAs, 75 s vs. 1,553 s at 1,000, and 141 s vs. 3,062 s at 2,000; Fig. S6A). The total memory footprint, summed over the whole process tree, is larger for siOFF: its mean peak proportional set size (PSS) is 40–45 GB, against 12–16 GB for the legacy pipeline (Fig. S6B). The benchmark dataset, genome indexing, expression annotation, and timing protocol are described in Sections S2.3 to S2.6.

S1.4 Accessibility pre-computation benchmark

Folding the complete GRCh38/hg38 reference (W=80, L=40, u=30, 64 workers on the AMD EPYC 9554 server) was compared between the legacy script (driven by GNU `parallel`) and siOFF's `accessibility` subcommand (Fig. S7, Table S1; implementation details in Section S2.7). siOFF completes the full-genome fold ∼23% faster (9.01 h vs. 11.75 h) and writes the same amount of data (192.6 GB for both), because both store one byte per position and unpaired-region length; siOFF's arrays are memory-mapped directly by the off-target stage (Section S2.7). The trade-off is a larger memory footprint: the peak proportional set size (PSS) summed over the whole process tree, sampled every 0.5 s, is ∼121 GB for siOFF against ∼18 GB for the legacy script (roughly 6.6×); this cost is incurred only once per genome and folding condition. Across all 910 chromosome–strand pairs (3,209,286,105 positions), the two sets of opening energies differ by at most 0.1 kcal/mol (one quantisation step of the one-byte encoding) at every position where both report a value.

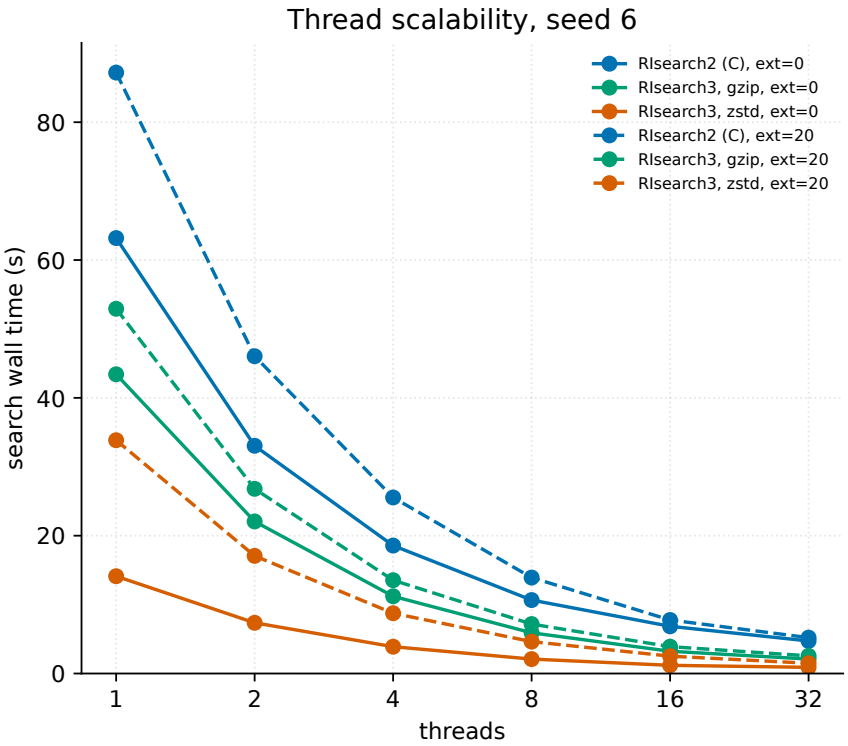


Figure S2: **Thread scaling of RIsearch versus RIsearch2** (seed scheme 6, extension lengths 0 and 20 nt, 1–32 threads). With matched gzip output, RIsearch is faster than RIsearch2 at every thread count in both extension modes, and its zstd output mode widens the gap further (RIsearch2 offers only gzip). Both tools scale near-linearly up to 16 threads.

Table S1: Full-genome accessibility pre-computation: legacy script versus siOFF's `accessibility` subcommand (hg38; W=80, L=40, u=30; 64 workers).

Metric	Legacy	New
Wall-clock time	11.75 h	9.01 h
Peak memory (summed PSS)	18.3 GB	121.0 GB
On-disk profile size	192.6 GB	192.6 GB
Files per chromosome	2	2 (+2 sidecars)

S2 Methods

S2.1 siOFF pipeline stages and partition-function model

The four-stage model of Alkan et al. (2017) is preserved. In the first stage, siRNA–target interaction predictions are generated by calling `risearch.index()` to build the suffix-array index of the target and `risearch.search()` to locate binding sites, both invoked directly through the Python bindings, returning results as a Polars DataFrame in the same process with no intermediate files on disk. In the second stage, predicted binding sites are intersected with an expression annotation (GTF, GFF3, or BED) to retain sites overlapping expressed features and to assign position-specific abundance values. Intersection is performed in-memory using an NCLS interval index (Alekseyenko and Lee, 2007) partitioned by chromosome and strand, with chromosome–strand pairs distributed across a thread pool; no external tools such as `bedtools` are needed. In the third stage, per-site binding-site opening energies $O_i = -RT \ln p_i$ are either computed on-the-fly from a genome FASTA or read from per-chromosome, per-strand profiles pre-computed by the `accessibility` subcommand, in both cases using the RNAplfold

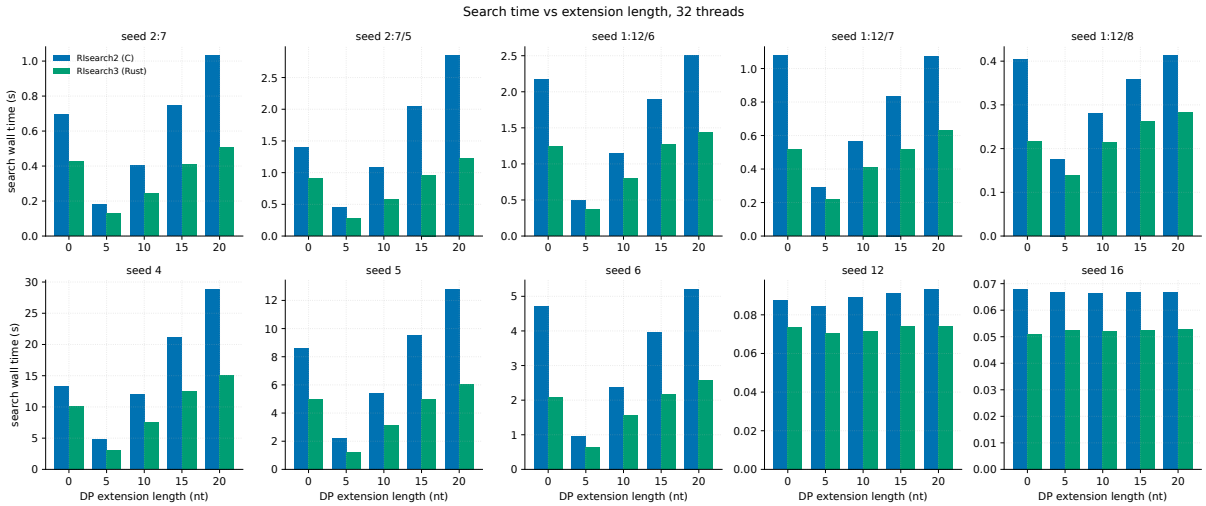


Figure S3: **Effect of extension length on search time** (0–20 nt, 32 threads, matched gzip output). Numbers above bars give RIsearch's time relative to RIsearch2. RIsearch's advantage grows with extension length as the Gotoh dynamic-programming stage comes to dominate runtime; at extension 0 (seed matching only) the two tools are closer.

algorithm (ViennaRNA 2.7.2 Python API (Lorenz et al., 2011); W=80, L=40, u=30) at the same temperature used for the partition function, so that E_i' and O_i are on a consistent thermodynamic scale. The Boltzmann-weighted sum

$$Z_s = \sum_{i \in \mathcal{I}_s} A_i \, \exp\left(\frac{-(E_i' + O_i)}{RT}\right) \tag{1}$$

is evaluated as a single vectorised `group_by` aggregation in Polars, where A_i is the transcript expression level, E_i' is the RIsearch hybridisation energy, $O_i = -RT \ln p_i$ is the opening energy derived from the RNAplfold unpaired probability p_i, $R = 1.987 \times 10^{-3}$ kcal mol^{-1} K^{-1} is the molar gas constant, and T is the folding temperature (default 310.15 K, 37 °C). When an on-target binding site is supplied via `--on-target-ids`, its Boltzmann weight W_{on} is computed identically and added to Z_s; the on-target probability $p_{\text{on}} = W_{\text{on}}/Z_s$ determines the aggregate off-targeting potential $P_{\text{OFF}} = 1 - p_{\text{on}}$. Per-transcript off-targeting probabilities $p_{\text{off},i} = W_i/Z_s$ are then computed as column operations on the resulting DataFrame. Parametric sweeps over α, γ, and θ are resolved in a single aggregation pass rather than repeated sequential iterations. In the fourth stage, results are written incrementally as each siRNA completes, in TSV, CSV, or Parquet format.

S2.2 Parallelism and memory

When processing a directory of per-siRNA prediction files, siOFF distributes work across a configurable process pool. A spawn-based multiprocessing context is used to avoid interaction with the Rayon thread pool used internally by Polars. The expression annotation is loaded once into Arrow IPC format and memory-mapped into each worker process via the operating-system page cache, keeping peak memory proportional to a single annotation copy regardless of worker count. Each worker performs the intersection, accessibility lookup, and partition-function evaluation for one siRNA at a time; prediction files are parsed lazily so that only filtered, selected columns are materialised, and output is streamed to disk incrementally throughout a batch run.

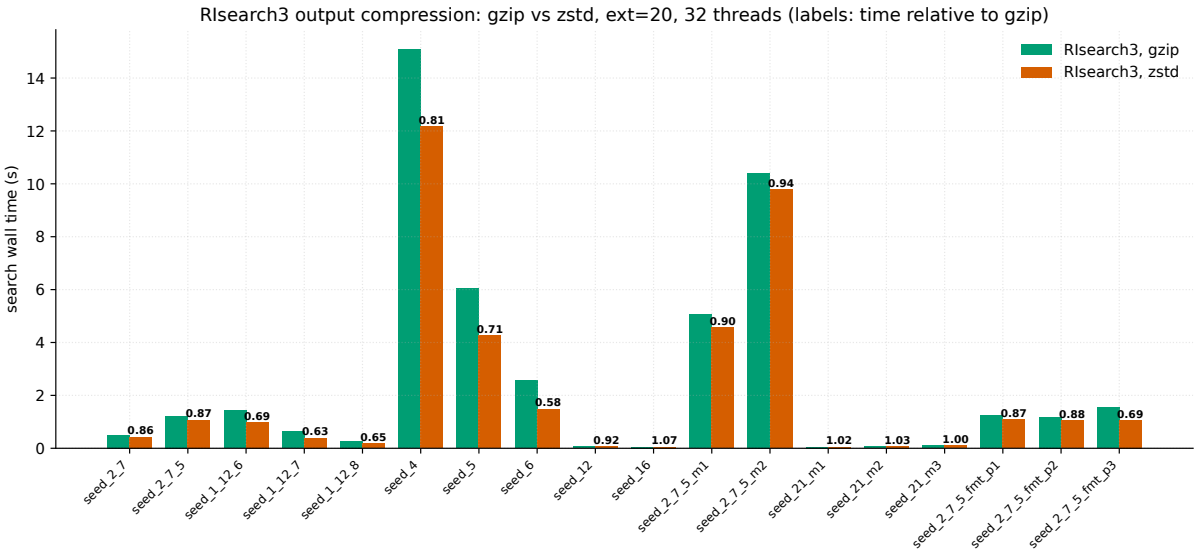


Figure S4: **Output-compression modes within RIsearch**: zstd versus gzip (32 threads, extension 20 nt). Numbers above bars give zstd time relative to gzip. On seed-and-extend workloads zstd trims up to ∼39% of wall-clock time (relative time 0.61–0.94) because compression, not search, is accelerated; on sub-second singleton-seed runs, where output is tiny, zstd can be marginally slower (up to 1.24). RIsearch2 offers only gzip-style output, so the head-to-head comparisons in Fig. S1 use gzip for both tools; with zstd, RIsearch's advantage over RIsearch2 widens further (Fig. S2).

S2.3 Benchmark dataset

The benchmark uses a subset of the genome-wide siRNA efficacy library of Huesken et al. (2005), which comprises 2,431 chemically synthesised 21-mer siRNAs targeting 34 human genes, with experimentally measured knockdown efficacies obtained in the H1299 non-small-cell lung carcinoma cell line. A master set of 2,000 siRNAs was drawn by uniform random sampling (without replacement) from this library; smaller batch sizes of 1,000 and 500 were produced by further subsampling from this master set, with a fresh random draw for each of the five benchmark iterations; the 2,000-siRNA batch is the master set itself and is identical across iterations. Target sequences for the off-target search were derived from the GRCh38/hg38 reference genome (UCSC assembly, December 2013). Expression values were taken from an H1299 RNA-seq experiment (ArrayExpress accession E-MTAB-2770), quantified per transcript (TPM) and attached as a `TPM` attribute to the transcript records of the GENCODE v47 GTF annotation (Section S2.5).

S2.4 Genome indexing and RIsearch2 binding-site search

The hg38 reference genome was indexed with RIsearch2 to produce a suffix-array package file (`hg38.pksuf`):

```
risearch2.x -c hg38.fa -o hg38.pksuf
```

where `-c` specifies the target FASTA to index and `-o` names the output suffix-array package. Binding-site predictions for each siRNA were generated once and stored as compressed output files; this search step was excluded from all pipeline benchmark timings so that only the post-search processing stages (intersection, accessibility annotation, partition function, and output) are measured.

The search command used was:

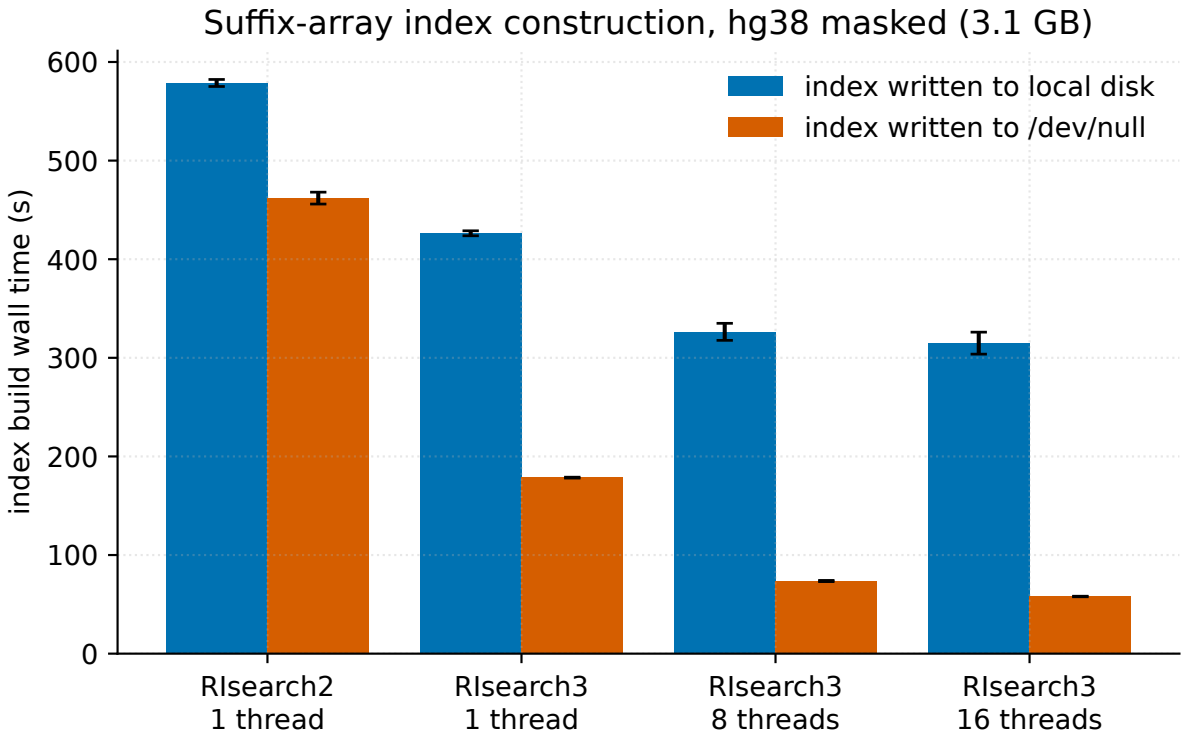


Figure S5: **Suffix-array index construction time for the masked GRCh38/hg38 genome (3.1 GB):** RIsearch2 (C, single-threaded) versus RIsearch3 at 1, 8 and 16 threads, with the index written either to local disk or to `/dev/null`. Error bars show the standard deviation across Hyperfine runs. Writing to local disk, RIsearch3 builds the index ∼1.9× faster at 16 threads (0.54× the RIsearch2 time); written to `/dev/null`, its parallel construction is up to ∼8.0× faster (0.13×), indicating that disk I/O dominates the write-to-disk timing.

```
risearch2.x \
    -i hg38.pksuf \
    -q huesken_on.fa \
    -s 2:7 \
    -e -10 \
    -p2 \
    -l 20 \
    -t 32
```

Key flag meanings:

- `-i hg38.pksuf` — pre-built suffix-array index of the hg38 target.
- `-q huesken_on.fa` — FASTA file of siRNA query sequences (guide strand, 5′→3′).
- `-s 2:7` — restrict seed matches to guide-strand positions 2–7 (the canonical seed region for RISC-mediated target recognition).
- `-e -10` — energy threshold of $-10\,\mathrm{kcal\,mol^{-1}}$; candidate sites with a hybridisation energy above this value are discarded.
- `-p2` — bindingsite output format, reporting one line per predicted interaction with genomic coordinates, energies, and alignment.

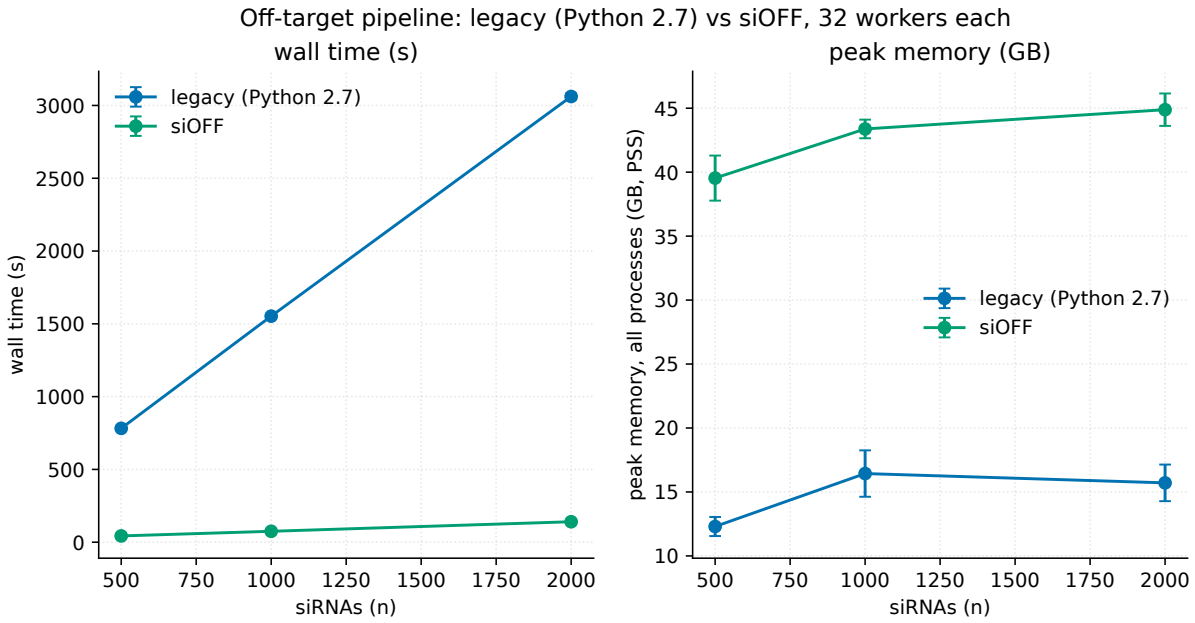


Figure S6: **Wall-clock time and peak memory of siOFF versus the legacy pipeline across batch sizes.** Points are the mean of five timed runs; error bars are 95% confidence intervals (Student's t). **(A)** Wall-clock time (seconds) for the intersection, accessibility annotation, Boltzmann partition-function, and output stages on 500, 1,000, and 2,000 siRNAs from the Hüsken et al. genome-wide efficacy library (Huesken et al., 2005). siOFF achieves an 18.1−21.8× speed-up relative to the legacy Python 2.7 implementation. **(B)** Peak memory (GB) of the whole process tree, measured as the summed proportional set size (PSS) of all processes, sampled every 0.5 s in each timed run. siOFF peaks at 40–45 GB, the legacy pipeline at 12–16 GB. Benchmarks were run on an AMD EPYC 9554 server (64 physical cores) with 32 workers for both pipelines; the same pre-computed RIsearch2 binding-site predictions were provided as input so that search time is excluded from both measurements.

- `-l 20` — maximum extension of the dynamic-programming alignment beyond the seed (20 nt), which keeps predicted target sites within the 30-nt span of the accessibility profiles.
- `-t 32` — use 32 parallel threads for the search.

S2.5 H1299 expression annotation

Per-transcript expression weights for the H1299 (NCI-H1299, large-cell lung carcinoma) cell line were derived from the RNA-seq dataset E-MTAB-2770 (*RNA-Seq of 675 human cancer cell lines*, CCLE), which is archived at ArrayExpress and contains data for approximately 675 cancer cell lines.

Sample identification. The SDRF file was queried to identify the single H1299 paired-end run, yielding accession `SRR8615898` (R1: 4.7 GB; R2: 4.8 GB).

Reference genome and annotation. GENCODE v47 was used as the reference (GRCh38 coordinate space): `gencode.v47.transcripts.fa.gz` for Salmon index construction and `gencode.v47.annotatio` for transcript coordinates.

Quantification. A Salmon v1.10.3 quasi-mapping index was built from the GENCODE transcript sequences with the `--gencode` flag (strips pipe-delimited header fields so transcript

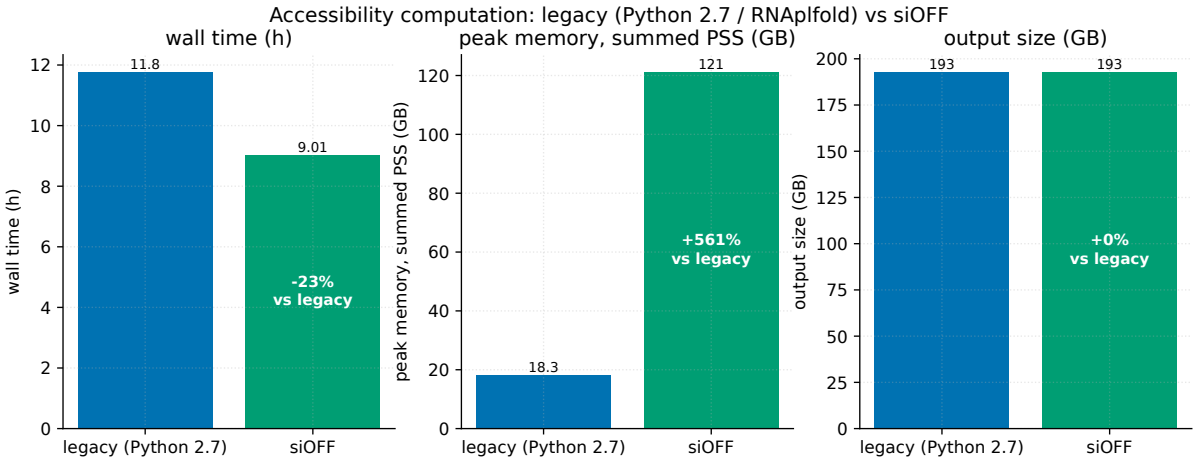


Figure S7: **Full-genome accessibility pre-computation: legacy script versus siOFF's `accessibility` subcommand** (hg38, W=80, L=40, u=30, 64 workers). **Left:** wall-clock time: siOFF folds the whole genome in 9.01 h versus 11.75 h for the legacy `RNAplfold` script (−23%). **Centre:** peak memory, measured as the proportional set size (PSS) summed over all processes and sampled every 0.5 s: 121 GB versus 18.3 GB for the legacy script. **Right:** total on-disk profile size across all chromosomes: 192.6 GB for both, as both store one byte per position and unpaired-region length. Values are also given in Table S1.

IDs match the GTF). Paired-end reads were quantified with:

```
salmon quant \
    -i input/salmon_index \
    -l A \
    -1 SRR8615898_1.fastq.gz \
    -2 SRR8615898_2.fastq.gz \
    -p 16 \
    --validateMappings \
    --gcBias \
    -o input/quant/SRR8615898
```

Key flags: `-l A` auto-detects library strandedness; `--validateMappings` applies alignment scoring to filter spurious mappings; `--gcBias` corrects for GC-content bias. The run achieved an 87.6% mapping rate.

BED7 construction. Salmon output (`quant.sf`, 384,354 transcript rows) was merged with GTF transcript-level coordinates using a custom script (`make_bed7.py`). Each GTF `transcript` record was converted to 0-based BED coordinates (BED start = GTF start −1; end unchanged) and joined to its TPM value by versioned ENST identifier. Transcripts absent from the Salmon output (PAR-locus duplicates and assembly patches) received TPM = 0. The output file (`annotation_E-MTAB-2770_GRCh38_v47.bed`) is a tab-separated BED7 file (385,659 rows; no header) in which column 7 carries the per-transcript TPM as a floating-point value:

chrom	start	end	transcript_id	.	strand	TPM
chr12	6534516	6538371	ENST00000229239.10	.	+	6130.06

Of the 385,659 transcripts, 143,881 (37.3%) carried TPM > 0, consistent with the fraction of GENCODE transcripts detectably expressed in a single cell-line condition. Transcripts with

TPM = 0 are retained in the file because the intersection service reads the annotation as a single indexed structure; dropping unexpressed entries would misalign row offsets. The high GAPDH TPM (ENST00000229239.10: 6,130) confirms biologically plausible quantification.

S2.6 Benchmark protocol

Timing and peak-memory measurements were collected on a server equipped with an AMD EPYC 9554 processor (64 physical cores, 128 threads) and sufficient RAM to keep all input files resident. Both pipelines used 32 parallel workers (`-j 32` for GNU parallel in the legacy case; `-j 32` process-pool workers for the new pipeline).

For each pipeline, one untimed warmup run on the 2,000-siRNA master set was executed and discarded to populate OS page-cache entries; then, for each batch size (500, 1,000, 2,000 siRNAs), five timed runs were executed, each with a fresh output directory. Wall-clock time was recorded with `/usr/bin/time -v` (GNU time). Because both pipelines spawn independent child processes (GNU parallel in the legacy case; a spawn-based process pool in siOFF), GNU time's maximum resident-set size reflects only the single largest process (∼8.8 GB for legacy, ∼4.7 GB for siOFF) rather than the run as a whole. Peak memory was therefore measured in every timed run by sampling the whole process tree every 0.5 s and recording the peak of the summed proportional set size (PSS), in which a page shared by N processes (e.g. a memory-mapped accessibility file) is charged $1/N$ to each.

For batch sizes of 1,000 and 500, each of the five timed runs used an independently drawn random subset of the master 2,000-siRNA set, ensuring that variance estimates reflect genuine sampling variability rather than run-to-run measurement noise alone. The same pre-computed RIsearch2 binding-site prediction files were provided as input to both pipelines, so that search time is excluded from all measurements. The legacy pipeline reads these gzip-compressed files directly; for siOFF they were converted once to Parquet, the format RIsearch (v3) emits natively, and this conversion is not timed. Likewise, siOFF memory-maps the per-chromosome accessibility arrays written by its `accessibility` subcommand (Section S2.7), analogous to the legacy pipeline's pre-computed binary accessibility files. Without accessibility, the off-targeting probabilities of the two pipelines agree for all 2,000 siRNAs of the master set (maximum absolute difference 3.9×10^{-4}). The results of this benchmark are presented in Section S1.3.

S2.7 Accessibility pre-computation

The legacy pipeline computed per-nucleotide RNA opening energies with a Python 2.7 script that wrapped the `RNAplfold` executable (ViennaRNA 2.5.1) as a subprocess, emitting two flat `uint8` binary files per chromosome (one per strand). siOFF's `accessibility` subcommand reproduces the same folding parameters (W=80, L=40, u=30) through the ViennaRNA 2.7.2 Python API directly, without spawning subprocesses: for each chromosome the call `RNA.fold_compound(...).probs_window(.` returns per-position unpaired-probability vectors at the specified temperature (default 37 °C, set via `--temperature`), from which opening energies $O_i = -RT \ln p_i$ are computed in-process and written, for each chromosome and strand, as a `uint8` NumPy array of opening energies ×10 (one byte per position and unpaired-region length, as in the legacy files), with a sidecar holding the exact values of entries above 25.5 kcal/mol; the off-target stage memory-maps these arrays without conversion. The corresponding full-genome benchmark is reported in Section S1.4.

S3 Temperature dependent energy parameters

Notation, N indicates an unknown nucleotide. In the stacking energies, the N is substituted with a nucleotide (ACGT/U) giving the stack the highest possible energy. _ indicates gap, a missing nucleotide, at the corresponding position, that is a bulge on the strand opposite the gap.

```
RNA: rA,rC,rG,rU,rN,r_
DNA: dA,dC,dG,dT,dN,d_

RNA/RNA: 5'-AC-3' bound to 3'-UG-5': rACUG
DNA/DNA: 5'-AC-3' bound to 3'-TG-5': dACTG
RNA/DNA: 5'-rAC-3' bound to 3'-dTG-5': rACdUG
```

The energy model of RIsearch is the sum of the energies of nucleotide stacks, where a stack is two nucleotides bound (aligned) to two nucleotides on the opposite strand or aligned to gaps on the opposite strand when the alignment contains bulges / gaps. A particular case is the stacks initiating or terminating the binding, which are written as, e.g. _G_C and G_C_, respectively. These stacks are not allowed in other places in the alignment.

The binding energy matrices are derived from those in ViennaRNA (Lorenz et al., 2011), which are again derived from the nearest neighbor energy model parameters from papers (Walter et al., 1994; Turner et al., 1988; Peritz et al., 1991; Jaeger et al., 1989; He et al., 1991; Turner and Mathews, 2010; Mathews et al., 1999, 2004) cited by ViennaRNA.

```
RNA/RNA: 2.6.3 RNAduplex

DNA/DNA: 2.6.3 RNAduplex -P DNA

RNA/DNA: 2.1.9h RNAduplex (in source code and binary only)
```

The RNA/RNA and DNA/DNA matrices are implemented in all later versions of ViennaRNA. However, the RNA/DNA matrices are only found in version 2.1.9h, where they are the default in certain programs including RNAduplex. The first (target) sequence must be the RNA, and the second the DNA, and these should be entered with the corresponding alphabets, that is with U or T, respectively.

When we allow GAPS along with the standard nucleotides, we have $5^4 = 625$ stacking energy parameters (SEP) we need to obtain values for in each of the four cases. However, not all SEPs are allowed, e.g. an alignment must start and end with a binding pair (_G_C is defined, but not _G_A) and a gap on both strands is not allowed. In addition, some SEPs are identical due to the nature of the underlying chemistry (rGCAU is the same as rUACG, but rGCdAT is not the same as rUAdCG) and some SEPs are linearly dependent due to the nature of the model (a gap opened must also be closed (an example gap opening could be rAAU_ and a closing could be rAA_U).

The undefined SEPs are set to a large positive energy (+20.0 kcal/mol) in the energy matrices, and we enforce linear dependencies according to the rules below

1. A stack of nucleotides is, e.g. rACGU or rACdGT is written as $(ijkl)$
2. Nucleotide are considered pairing when one of AU, AT, CG, GU and GT
3. If i is paired but j is not paired: $(ijkl) = (jilk)$, e.g. rACdGG = rCAdGG
4. For RNA/RNA and DNA/DNA $(ijkl) = (lkji)$
5. For unknown nucleotides (N) the maximum (worst) value is chosen amongst the possibilities for N. In this way we determine the $6^4 = 1296$ SEPs needed for RIsearch.

These rules reduced the number of parameters to be determined from 625 to 266 for RNA/RNA and DNA/DNA and to 459 in RNA/DNA and removed the linear interdependency of the parameters. The actual number of SEPs to be determined is a little lower when illegal stacking values are removed.

The SEPs were obtained by fitting energies and alignments from the respective RNA, DNA and hybrid modes of the RNAduplex according to the list above. To this end we constructed 4 times 50,000 alignments of the following types

1. Exact complementary sequences
2. GU internal size 1–4 and, terminal, size -4
3. internal loops of size 1–4
4. internal bulges of size 1–4

In each case the sequences were otherwise random and constructed to be 10–20 nt complementary + variation 4 nt + 10–20 nt complementary. The alignments obtained from RNAduplex were then parameterized according to the corresponding stacking base pairs and the SEPs were obtained by a linear regression fit of the RNAduplex energy. In cases where the alignments were not full length, the alignments and energies were discarded before fitting.

The RNA/DNA parameters in both the previous versions of RIsearch and in the 2.1.9h version of ViennaRNA were constructed from the Sugimoto 95 paper (Sugimoto et al., 1995), which only contains the SEPs for canonical basepair stacks like rACdTG. Missing values were obtained as the average of corresponding RNA/RNA and DNA/DNA SEP. However the RNA/DNA parameters have been improved upon in two papers (Sugimoto et al., 2000; Watkins et al., 2011), which respectively adds temperature dependent SEPs for mismatches with neighbouring CG pairs, e.g. rGCGdCAC and for symmetrical mismatches, e.g. rAGdTG and rGGdCG. These values have been used in place of the values obtained by fitting in the final RNA/DNA SEP matrix.

References

A. V. Alekseyenko and C. J. Lee. Nested Containment List (NCList): a new algorithm for accelerating interval query of genome alignment and interval databases. *Bioinformatics*, 23 (11):1386–1393, 2007. doi: 10.1093/bioinformatics/btl647.

F. Alkan, A. Wenzel, O. Palasca, P. Kerpedjiev, A. F. Rudebeck, P. F. Stadler, I. L. Hofacker, and J. Gorodkin. RIsearch2: suffix array-based large-scale prediction of RNA–RNA interactions and siRNA off-targets. *Nucleic Acids Research*, 45(8):e60, 2017. doi: 10.1093/nar/gkw1325.

I. Grebnov. libsais: linear-time suffix array, LCP array and Burrows–Wheeler transform construction. https://github.com/IlyaGrebnov/libsais, 2021. Accessed 2026.

L. He, R. Kierzek, J. SantaLucia, Jr., A. E. Walter, and D. H. Turner. Nearest-neighbor parameters for G·U mismatches: 5'gu3'/3'ug5' is destabilizing in the contexts cgug/gugc, ugua/augu, and aguu/uuga but stabilizing in GGUC/CUGG. *Biochemistry*, 30(46):11124–11132, 1991. doi: 10.1021/bi00110a015.

D. Huesken, J. Lange, C. Mickanin, J. Weiler, F. Asselbergs, J. Warner, B. Meloon, S. Engel, A. Rosenberg, D. Cohen, M. Labow, M. Reinhardt, F. Natt, and J. Hall. Design of a genome-wide siRNA library using an artificial neural network. *Nature Biotechnology*, 23(8):995–1001, 2005. doi: 10.1038/nbt1118.

J. A. Jaeger, D. H. Turner, and M. Zuker. Improved predictions of secondary structures for RNA. *Proceedings of the National Academy of Sciences USA*, 86(20):7706–7710, 1989. doi: 10.1073/pnas.86.20.7706.

R. Lorenz, S. H. Bernhart, C. Höner zu Siederdissen, H. Tafer, C. Flamm, P. F. Stadler, and I. L. Hofacker. ViennaRNA package 2.0. *Algorithms for Molecular Biology*, 6(1):26, 2011. doi: 10.1186/1748-7188-6-26.

D. H. Mathews, J. Sabina, M. Zuker, and D. H. Turner. Expanded sequence dependence of thermodynamic parameters improves prediction of RNA secondary structure. *Journal of Molecular Biology*, 288(5):911–940, 1999. doi: 10.1006/jmbi.1999.2700.

D. H. Mathews, M. D. Disney, J. L. Childs, S. J. Schroeder, M. Zuker, and D. H. Turner. Incorporating chemical modification constraints into a dynamic programming algorithm for prediction of RNA secondary structure. *Proceedings of the National Academy of Sciences USA*, 101(19):7287–7292, 2004. doi: 10.1073/pnas.0401799101.

A. E. Peritz, R. Kierzek, N. Sugimoto, and D. H. Turner. Thermodynamic study of internal loops in oligoribonucleotides: symmetric loops are more stable than asymmetric loops. *Biochemistry*, 30(26):6428–6436, 1991. doi: 10.1021/bi00240a013.

N. Sugimoto, S.-i. Nakano, M. Katoh, A. Matsumura, H. Nakamuta, T. Ohmichi, M. Yoneyama, and M. Sasaki. Thermodynamic parameters to predict stability of RNA/DNA hybrid duplexes. *Biochemistry*, 34(35):11211–11216, 1995. doi: 10.1021/bi00035a029.

N. Sugimoto, M. Nakano, and S.-i. Nakano. Thermodynamics–structure relationship of single mismatches in RNA/DNA duplexes. *Biochemistry*, 39(37):11270–11281, 2000. doi: 10.1021/bi000819p.

D. H. Turner and D. H. Mathews. NNDB: the nearest neighbor parameter database for predicting stability of nucleic acid secondary structure. *Nucleic Acids Research*, 38(suppl_1):D280–D282, 2010. doi: 10.1093/nar/gkp892.

D. H. Turner, N. Sugimoto, and S. M. Freier. RNA structure prediction. *Annual Review of Biophysics and Biophysical Chemistry*, 17(1):167–192, 1988. doi: 10.1146/annurev.bb.17.060188.001123.

A. E. Walter, D. H. Turner, J. Kim, M. H. Lyttle, P. Müller, D. H. Mathews, and M. Zuker. Coaxial stacking of helixes enhances binding of oligoribonucleotides and improves predictions of RNA folding. *Proceedings of the National Academy of Sciences USA*, 91(20):9218–9222, 1994. doi: 10.1073/pnas.91.20.9218.

N. E. Watkins, Jr., W. J. Kennelly, M. J. Tsay, A. Tuin, L. Swenson, H.-R. Lee, S. Morosyuk, D. A. Hicks, and J. SantaLucia, Jr. Thermodynamic contributions of single internal ra·da, rc·dc, rg·dg and ru·dt mismatches in RNA/DNA duplexes. *Nucleic Acids Research*, 39(5):1894–1902, 2011. doi: 10.1093/nar/gkq905.